

Woxsen University

School of Technology



PROJECT REPORT

on

CROP PLANNING & FARM INPUT MANAGEMENT SYSTEM

Design and Implementation of a Database Management System

Name: Sufiyan Yusuf Jahan

Roll Number: 25WU0102276

Team: AIML Whales

Course: Database Management Systems (DBMS)

Faculty Name: Dr. Kiran Mayee

Academic Year: 2026–27

ABSTRACT

Farms run many connected activities: they hold land in fields, plan crops by season, apply inputs, irrigate, employ labour, harvest and sell. Kept in separate sheets, this data duplicates easily and is hard to report on. This project presents a relational database for crop planning and farm input management.

The ER model has 14 entities and 13 relationships and is mapped to 14 tables in third normal form. Integrity is enforced with primary and foreign keys, 19 CHECK constraints, 2 UNIQUE keys, a generated column and 3 triggers for cross-table rules.

The database was implemented in MySQL, loaded with 46 rows of consistent sample data, and queried with joins, aggregates, a nested query and the cropprofitability view. Constraint and trigger tests verified the rules. A Streamlit prototype demonstrates viewing, inserting and deleting Farm records.

Keywords: DBMS, MySQL, ER Model, Normalization, Triggers, Views, Crop Planning, Farm Inputs, Streamlit

TABLE OF CONTENTS

1. INTRODUCTION AND PROBLEM STATEMENT.....	1
1.1 Introduction.....	1
1.2 Problem Statement.....	1
2. OBJECTIVES AND SCOPE.....	1
2.1 Objectives.....	1
2.2 Scope.....	1
2.3 Limitations.....	1
3. SOFTWARE AND HARDWARE REQUIREMENTS.....	2
3.1 Software Requirements.....	2
3.2 Hardware Requirements.....	2
4. ER DIAGRAM.....	3
4.1 Core and Planning Layer.....	3
4.2 Operations and Outcomes Layer.....	3
4.3 Relationships and Cardinality.....	4
5. RELATIONAL SCHEMA AND NORMALIZATION.....	5
5.1 Relational Schema.....	5
5.2 Normalization.....	5
5.2.1 First Normal Form (1NF).....	5
5.2.2 Second Normal Form (2NF).....	6
5.2.3 Third Normal Form (3NF).....	6
5.2.4 Boyce-Codd Normal Form (BCNF).....	6
6. DATA DICTIONARY.....	7
7. SQL COMMANDS USED (DDL AND DML) WITH SAMPLE OUTPUTS.....	12
7.1 Data Definition Language (DDL).....	12
7.2 Constraints Summary.....	12
7.3 Triggers.....	13
7.4 Data Manipulation Language (DML).....	13
7.5 Sample Outputs.....	14
8. QUERIES WITH OUTPUTS.....	16
9. UI DESIGN AND SCREENSHOTS.....	19
9.1 Overview.....	19
9.2 View Farm Records.....	19
9.3 Insert a Farm Record.....	19
9.4 Delete a Farm Record.....	20
10. IMPLEMENTATION DETAILS.....	21
10.1 Technology Stack.....	21
10.2 Database Connectivity.....	21
10.3 Key Code Snippets.....	21
10.4 Integrity Features.....	22
11. TESTING.....	23
11.1 Test Cases and Results.....	23
11.2 Observations.....	24

12. CONCLUSION AND FUTURE ENHANCEMENTS.....	25
12.1 Conclusion.....	25
12.2 Future Enhancements.....	25
13. REFERENCES.....	25
APPENDIX: GITHUB REPOSITORY LINK.....	25

1. INTRODUCTION AND PROBLEM STATEMENT

1.1 Introduction

Farm data is connected: land, seasons, crops, inputs, labour, harvests and sales. A relational database stores it once and keeps it consistent through keys and constraints [1]. The design process followed was the ER model [5], relational schema [4], normalization, SQL implementation and a UI prototype.

1.2 Problem Statement

This project supports crop planning and tracked input management for small-to-medium farms. Manual records cause these problems:

- Crop, variety and field details are duplicated
- Inputs, irrigation and labour are not tied to the crop plan that used them
- Cost per plan and profitability are hard to calculate
- Sales can exceed what was harvested
- Activities can fall outside season or plan dates

2. OBJECTIVES AND SCOPE

2.1 Objectives

1. Build an ER model of the farm lifecycle.
2. Map it to a 14-table relational schema.
3. Normalize the schema to 3NF.
4. Implement it in MySQL with constraints.
5. Enforce business rules with CHECK constraints and triggers.
6. Populate it with sample data.
7. Demonstrate it with joins, aggregates, a nested query and a view.
8. Build a Streamlit prototype and verify integrity through tests.

2.2 Scope

The scope covers farm setup (Farm, Field, Season, SoilTest), catalog (Crop, Variety, Input), planning (CropPlan), operations (Application, IrrigationEvent, LabourActivity), outcomes (Harvest, Buyer, Sale), reporting (view and queries), and a Farm-focused UI.

2.3 Limitations

- The UI demonstrates CRUD mainly through the Farm entity, not every entity
- The application runs locally and needs a running local MySQL server
- Credentials are kept in environment variables and are not committed to GitHub
- There is no user authentication or role-based access control
- It is a functional academic prototype, not a production deployment
- Triggers fire on INSERT only, so an UPDATE can move a CropPlan outside its Season dates
- IrrigationEvent and LabourActivity dates are not checked against the plan window
- The database does not enforce that field areas fit within the farm area, or planned area within the field area

3. SOFTWARE AND HARDWARE REQUIREMENTS

3.1 Software Requirements

Component	Requirement
Database management system	MySQL 9.7.1
Database client	MySQL command-line client / MySQL Workbench
Operating system	macOS
Front end / back end	Python, Streamlit, mysql-connector-python, python-dotenv
Design tool	Claude
Version control	Git and GitHub

Table 3.1: Software Requirements

3.2 Hardware Requirements

Component	Minimum requirement
Processor	Dual-core processor, 2 GHz or higher

Component	Minimum requirement
Memory (RAM)	4 GB (8 GB recommended)
Storage	500 MB free disk space
Display	1366 × 768 resolution or higher

Table 3.2: Hardware Requirements

4. ER DIAGRAM

The refined ER model has 14 entities and 13 relationships. For readability it is drawn in two parts (Figures 4.1 and 4.2). The model has three layers: Core (Farm, Field, Season), Catalog (Crop, Variety, SoilTest, Input) and Operations & Outcomes (CropPlan, Application, IrrigationEvent, LabourActivity, Harvest, Buyer, Sale). Rectangles are entities, diamonds are relationships and ovals are attributes, with PK/FK and 1/N cardinality.

4.1 Core and Planning Layer

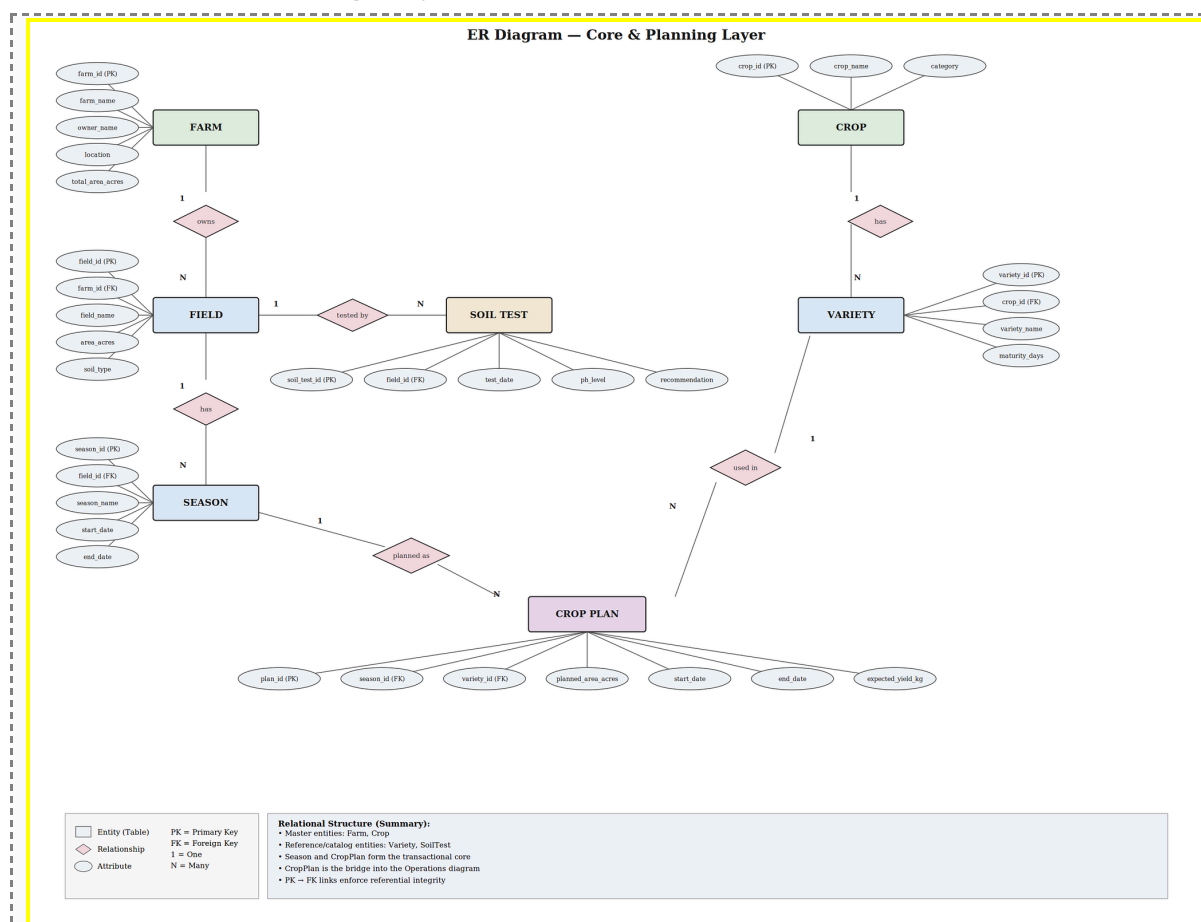


Figure 4.1: ER Diagram Part 1 – Core and Planning Layer

4.2 Operations and Outcomes Layer

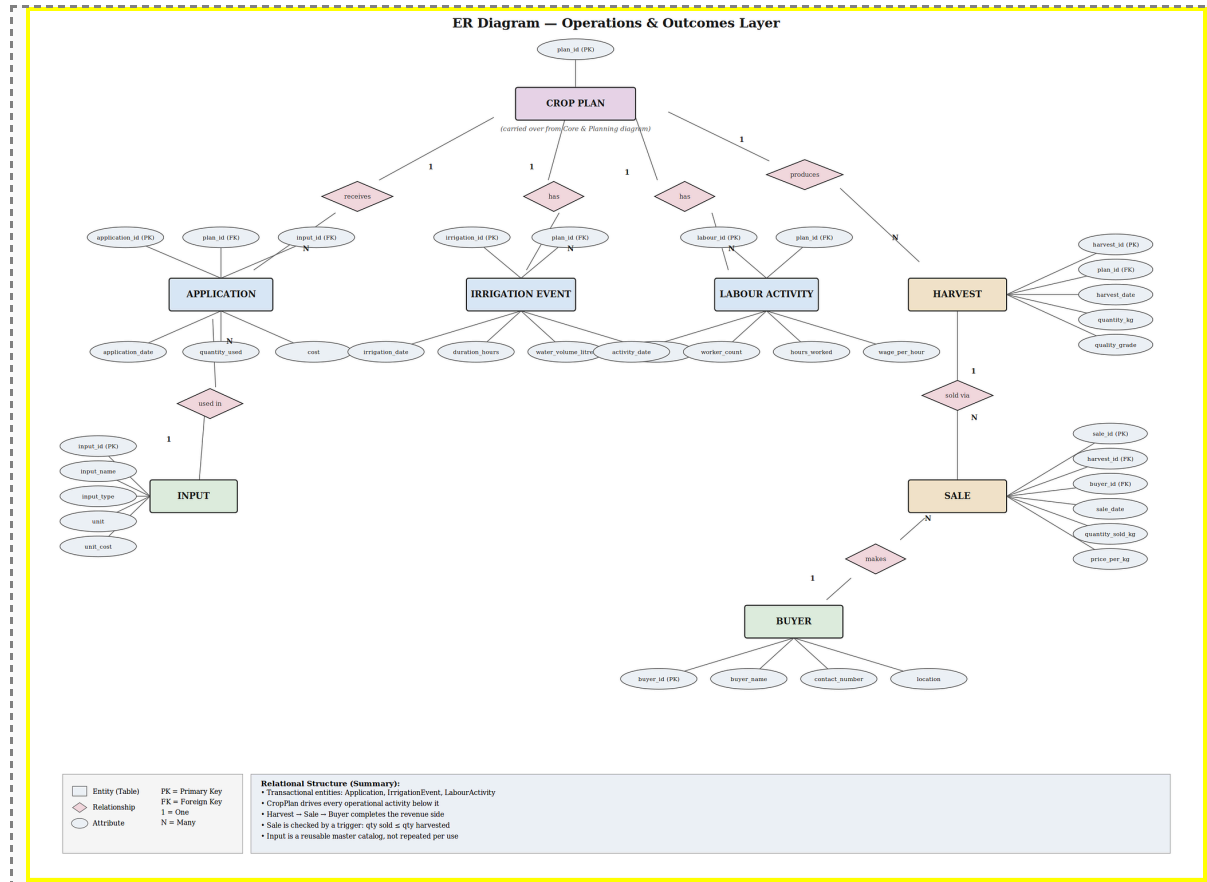


Figure 4.2: ER Diagram Part 2 – Operations and Outcomes Layer

4.3 Relationships and Cardinality

Relationship	Entities	Cardinality	On delete
OWNS	Farm – Field	1 : N	CASCADE
HAS	Field – Season	1 : N	CASCADE
TESTED BY	Field – SoilTest	1 : N	CASCADE
PLANNED AS	Season – CropPlan	1 : N	CASCADE
HAS	Crop – Variety	1 : N	CASCADE
USED IN	Variety – CropPlan	1 : N	RESTRICT
RECEIVES	CropPlan – Application	1 : N	CASCADE
USED IN	Input – Application	1 : N	RESTRICT
HAS	CropPlan – IrrigationEvent	1 : N	CASCADE
HAS	CropPlan – LabourActivity	1 : N	CASCADE
PRODUCES	CropPlan – Harvest	1 : N	CASCADE
SOLD VIA	Harvest – Sale	1 : N	CASCADE

Relationship	Entities	Cardinality	On delete
MAKES	Buyer – Sale	1 : N	RESTRICT

Table 4.1: Relationships and Cardinality

CropPlan and Input are in a many-to-many relationship, resolved by the Application table. Harvest and Buyer are also many-to-many, resolved by the Sale table. Both resolving tables carry their own attributes.

5. RELATIONAL SCHEMA AND NORMALIZATION

5.1 Relational Schema

Every entity maps to one table, and foreign keys carry the cardinalities (Table 5.1). PK is a primary key, FK a foreign key and UQ a unique key.

Relation	Attributes	Keys
Farm	farm_id, farm_name, owner_name, location, total_area_acres	PK farm_id
Field	field_id, farm_id, field_name, area_acres, soil_type	PK field_id; FK farm_id
Season	season_id, field_id, season_name, start_date, end_date	PK season_id; FK field_id
Crop	crop_id, crop_name, category	PK crop_id; UQ crop_name
Variety	variety_id, crop_id, variety_name, maturity_days	PK variety_id; FK crop_id; UQ (crop_id, variety_name)
SoilTest	soil_test_id, field_id, test_date, ph_level, nitrogen_level, phosphorus_level, potassium_level, recommendation	PK soil_test_id; FK field_id
CropPlan	plan_id, season_id, variety_id, planned_area_acres, start_date, end_date, expected_yield_kg	PK plan_id; FK season_id, variety_id
Input	input_id, input_name, input_type, unit, unit_cost	PK input_id
Application	application_id, plan_id, input_id, application_date, quantity_used, cost	PK application_id; FK plan_id, input_id
IrrigationEvent	irrigation_id, plan_id, irrigation_date, duration_hours, water_volume_litres, method	PK irrigation_id; FK plan_id
LabourActivity	labour_id, plan_id, activity_date, task_description, worker_count, hours_worked, wage_per_hour	PK labour_id; FK plan_id
Harvest	harvest_id, plan_id, harvest_date, quantity_kg, quality_grade, storage_location	PK harvest_id; FK plan_id
Buyer	buyer_id, buyer_name, contact_number, location	PK buyer_id
Sale	sale_id, harvest_id, buyer_id, sale_date, quantity_sold_kg, price_per_kg, total_amount	PK sale_id; FK harvest_id, buyer_id

Table 5.1: Relational Schema

5.2 Normalization

5.2.1 First Normal Form (1NF)

All columns are atomic with no repeating groups; each Variety row holds one variety.

5.2.2 Second Normal Form (2NF)

Every table has a single-column surrogate key (`INT AUTO_INCREMENT`), so partial dependencies cannot occur.

5.2.3 Third Normal Form (3NF)

Crop and variety facts live only in Crop and Variety. Input details live in Input, not in each Application. Field details are reached through Season, so CropPlan has no `field_id` and no transitive redundancy.

Derived values: `Application.cost` is stored (`quantity_used × Input.unit_cost`) as a deliberate price snapshot, since `unit_cost` can change later. `Sale.total_amount` is a STORED generated column, so the database computes it and it cannot drift.

5.2.4 Boyce-Codd Normal Form (BCNF)

Alternate keys `crop_name` and (`crop_id`, `variety_name`) are candidate keys.

6. DATA DICTIONARY

The data dictionary lists the structure of each of the 14 tables: column, data type, constraints and meaning. Every table uses an INT AUTO_INCREMENT primary key.

Column	Data Type	Constraints	Description
farm_id	INT	PK, AUTO_INCREMENT	Unique farm identifier
farm_name	VARCHAR(100)	NOT NULL	Name of the farm
owner_name	VARCHAR(100)	NOT NULL	Name of the owner
location	VARCHAR(150)	NOT NULL	Location of the farm
total_area_acres	DECIMAL(8,2)	NOT NULL, CHECK > 0	Total farm area in acres

Table 6.1: Farm

Column	Data Type	Constraints	Description
field_id	INT	PK, AUTO_INCREMENT	Unique field identifier
farm_id	INT	FK → Farm, NOT NULL, ON DELETE CASCADE	Farm that owns the field
field_name	VARCHAR(100)	NOT NULL	Name of the field
area_acres	DECIMAL(8,2)	NOT NULL, CHECK > 0	Area of the field in acres
soil_type	VARCHAR(50)	NULL	Type of soil in the field

Table 6.2: Field

Column	Data Type	Constraints	Description
season_id	INT	PK, AUTO_INCREMENT	Unique season identifier
field_id	INT	FK → Field, NOT NULL, ON DELETE CASCADE	Field the season belongs to
season_name	VARCHAR(50)	NOT NULL	Name of the season
start_date	DATE	NOT NULL	Season start date
end_date	DATE	NOT NULL, CHECK end_date > start_date	Season end date

Table 6.3: Season

Column	Data Type	Constraints	Description
crop_id	INT	PK, AUTO_INCREMENT	Unique crop identifier
crop_name	VARCHAR(50)	NOT NULL, UNIQUE	Name of the crop
category	VARCHAR(50)	NULL	Category of the crop

Table 6.4: Crop

Column	Data Type	Constraints	Description
variety_id	INT	PK, AUTO_INCREMENT	Unique variety identifier
crop_id	INT	FK → Crop, NOT NULL, ON DELETE CASCADE	Crop the variety belongs to
variety_name	VARCHAR(50)	NOT NULL; UNIQUE (crop_id, variety_name)	Name of the variety
maturity_days	INT	NOT NULL, CHECK > 0	Days the variety takes to mature

Table 6.5: Variety

Column	Data Type	Constraints	Description
soil_test_id	INT	PK, AUTO_INCREMENT	Unique soil test identifier
field_id	INT	FK → Field, NOT NULL, ON DELETE CASCADE	Field that was tested
test_date	DATE	NOT NULL	Date of the test
ph_level	DECIMAL(3,1)	CHECK BETWEEN 0 AND 14	Soil pH value
nitrogen_level	VARCHAR(20)	NULL	Nitrogen level
phosphorus_level	VARCHAR(20)	NULL	Phosphorus level
potassium_level	VARCHAR(20)	NULL	Potassium level
recommendation	VARCHAR(255)	NULL	Recommended action

Table 6.6: SoilTest

Column	Data Type	Constraints	Description
plan_id	INT	PK, AUTO_INCREMENT	Unique crop plan identifier
season_id	INT	FK → Season, NOT NULL, ON DELETE CASCADE	Season of the plan
variety_id	INT	FK → Variety, NOT NULL, ON DELETE RESTRICT	Variety to be grown
planned_area_acres	DECIMAL(8,2)	NOT NULL, CHECK > 0	Area planned for the crop
start_date	DATE	NOT NULL	Plan start date
end_date	DATE	NOT NULL, CHECK end_date > start_date	Plan end date
expected_yield_kg	DECIMAL(10,2)	NULL, CHECK >= 0	Expected yield in kg

Table 6.7: CropPlan

Column	Data Type	Constraints	Description
input_id	INT	PK, AUTO_INCREMENT	Unique input identifier

Column	Data Type	Constraints	Description
input_name	VARCHAR(100)	NOT NULL	Name of the input
input_type	VARCHAR(50)	NOT NULL	Type of input
unit	VARCHAR(20)	NOT NULL	Unit of measurement
unit_cost	DECIMAL(10,2)	NOT NULL, CHECK > 0	Cost per unit

Table 6.8: Input

Column	Data Type	Constraints	Description
application_id	INT	PK, AUTO_INCREMENT	Unique application identifier
plan_id	INT	FK → CropPlan, NOT NULL, ON DELETE CASCADE	Plan the input was applied to
input_id	INT	FK → Input, NOT NULL, ON DELETE RESTRICT	Input that was applied
application_date	DATE	NOT NULL (trigger-checked)	Date of application
quantity_used	DECIMAL(10,2)	NOT NULL, CHECK > 0	Quantity applied
cost	DECIMAL(10,2)	NOT NULL, CHECK >= 0	Cost snapshot (quantity × unit cost)

Table 6.9: Application

Column	Data Type	Constraints	Description
irrigation_id	INT	PK, AUTO_INCREMENT	Unique irrigation identifier
plan_id	INT	FK → CropPlan, NOT NULL, ON DELETE CASCADE	Plan that was irrigated
irrigation_date	DATE	NOT NULL	Date of irrigation
duration_hours	DECIMAL(4,1)	NOT NULL, CHECK > 0	Duration in hours
water_volume_litres	DECIMAL(10,2)	NULL, CHECK >= 0	Water used in litres
method	VARCHAR(30)	NULL	Irrigation method

Table 6.10: IrrigationEvent

Column	Data Type	Constraints	Description
labour_id	INT	PK, AUTO_INCREMENT	Unique labour activity identifier
plan_id	INT	FK → CropPlan, NOT NULL, ON DELETE CASCADE	Plan the work was done for
activity_date	DATE	NOT NULL	Date of the activity
task_description	VARCHAR(100)	NOT NULL	Task carried out
worker_count	INT	NOT NULL, CHECK > 0	Number of workers

Column	Data Type	Constraints	Description
hours_worked	DECIMAL(5,1)	NOT NULL, CHECK > 0	Hours worked
wage_per_hour	DECIMAL(8,2)	NOT NULL, CHECK > 0	Wage per hour

Table 6.11: LabourActivity

Column	Data Type	Constraints	Description
harvest_id	INT	PK, AUTO_INCREMENT	Unique harvest identifier
plan_id	INT	FK → CropPlan, NOT NULL, ON DELETE CASCADE	Plan that produced the harvest
harvest_date	DATE	NOT NULL	Date of harvest
quantity_kg	DECIMAL(10,2)	NOT NULL, CHECK >= 0	Quantity harvested in kg
quality_grade	VARCHAR(10)	NULL	Quality grade
storage_location	VARCHAR(100)	NULL	Where the harvest is stored

Table 6.12: Harvest

Column	Data Type	Constraints	Description
buyer_id	INT	PK, AUTO_INCREMENT	Unique buyer identifier
buyer_name	VARCHAR(100)	NOT NULL	Name of the buyer
contact_number	VARCHAR(15)	NULL	Contact number
location	VARCHAR(150)	NULL	Location of the buyer

Table 6.13: Buyer

Column	Data Type	Constraints	Description
sale_id	INT	PK, AUTO_INCREMENT	Unique sale identifier
harvest_id	INT	FK → Harvest, NOT NULL, ON DELETE CASCADE	Harvest that was sold
buyer_id	INT	FK → Buyer, NOT NULL, ON DELETE RESTRICT	Buyer of the sale
sale_date	DATE	NOT NULL	Date of sale
quantity_sold_kg	DECIMAL(10,2)	NOT NULL, CHECK > 0	Quantity sold in kg
price_per_kg	DECIMAL(8,2)	NOT NULL, CHECK > 0	Price per kg
total_amount	DECIMAL(12,2)	GENERATED ALWAYS AS (quantity_sold_kg * price_per_kg) STORED	Computed sale amount

Table 6.14: Sale

7. SQL COMMANDS USED (DDL AND DML) WITH SAMPLE OUTPUTS

7.1 Data Definition Language (DDL)

The database and its 14 tables are created with CREATE DATABASE and CREATE TABLE statements, in dependency order: Farm, Field, Season, Crop, Variety, SoilTest, CropPlan, Input, Application, IrrigationEvent, LabourActivity, Harvest, Buyer, Sale.

```

CREATE TABLE `Farm` (
  `farm_id` int NOT NULL AUTO_INCREMENT,
  `farm_name` varchar(100) NOT NULL,
  `owner_name` varchar(100) NOT NULL,
  `location` varchar(150) NOT NULL,
  `total_area_acres` decimal(8,2) NOT NULL,
  PRIMARY KEY (`farm_id`),
  CONSTRAINT `farm_chk_1` CHECK ((`total_area_acres` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `Field` (
  `field_id` int NOT NULL AUTO_INCREMENT,
  `farm_id` int NOT NULL,
  `field_name` varchar(100) NOT NULL,
  `area_acres` decimal(8,2) NOT NULL,
  `soil_type` varchar(50) DEFAULT NULL,
  PRIMARY KEY (`field_id`),
  KEY `farm_id` (`farm_id`),
  CONSTRAINT `field_ibfk_1`
    FOREIGN KEY (`farm_id`) REFERENCES `Farm` (`farm_id`) ON DELETE CASCADE,
  CONSTRAINT `field_chk_1` CHECK ((`area_acres` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `Season` (
  `season_id` int NOT NULL AUTO_INCREMENT,
  `field_id` int NOT NULL,
  `season_name` varchar(50) NOT NULL,
  `start_date` date NOT NULL,
  `end_date` date NOT NULL,
  PRIMARY KEY (`season_id`),
  KEY `field_id` (`field_id`),
  CONSTRAINT `season_ibfk_1`
    FOREIGN KEY (`field_id`) REFERENCES `Field` (`field_id`) ON DELETE CASCADE,
  CONSTRAINT `season_chk_1` CHECK ((`end_date` > `start_date`))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `Crop` (
  `crop_id` int NOT NULL AUTO_INCREMENT,
  `crop_name` varchar(50) NOT NULL,
  `category` varchar(50) DEFAULT NULL,
  PRIMARY KEY (`crop_id`),
  UNIQUE KEY `crop_name` (`crop_name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `Variety` (
  `variety_id` int NOT NULL AUTO_INCREMENT,
  `crop_id` int NOT NULL,
  `variety_name` varchar(50) NOT NULL,
  `maturity_days` int NOT NULL,
  PRIMARY KEY (`variety_id`),
  UNIQUE KEY `crop_id` (`crop_id`,`variety_name`),
  CONSTRAINT `variety_ibfk_1`
    FOREIGN KEY (`crop_id`) REFERENCES `Crop` (`crop_id`) ON DELETE CASCADE,
  CONSTRAINT `variety_chk_1` CHECK ((`maturity_days` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

Listing 7.1: DDL for Farm, Field, Season, Crop and Variety

```

CREATE TABLE `SoilTest` (
  `soil_test_id` int NOT NULL AUTO_INCREMENT,
  `field_id` int NOT NULL,
  `test_date` date NOT NULL,
  `ph_level` decimal(3,1) DEFAULT NULL,
  `nitrogen_level` varchar(20) DEFAULT NULL,
  `phosphorus_level` varchar(20) DEFAULT NULL,
  `potassium_level` varchar(20) DEFAULT NULL,
  `recommendation` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`soil_test_id`),
  KEY `field_id` (`field_id`),
  CONSTRAINT `soiltest_ibfk_1`
    FOREIGN KEY (`field_id`) REFERENCES `Field` (`field_id`) ON DELETE CASCADE,
  CONSTRAINT `soiltest_chk_1` CHECK ((`ph_level` between 0 and 14))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `CropPlan` (
  `plan_id` int NOT NULL AUTO_INCREMENT,
  `season_id` int NOT NULL,
  `variety_id` int NOT NULL,
  `planned_area_acres` decimal(8,2) NOT NULL,
  `start_date` date NOT NULL,
  `end_date` date NOT NULL,
  `expected_yield_kg` decimal(10,2) DEFAULT NULL,
  PRIMARY KEY (`plan_id`),
  KEY `season_id` (`season_id`),
  KEY `variety_id` (`variety_id`),
  CONSTRAINT `cropplan_ibfk_1`
    FOREIGN KEY (`season_id`) REFERENCES `Season` (`season_id`) ON DELETE CASCADE,
  CONSTRAINT `cropplan_ibfk_2`
    FOREIGN KEY (`variety_id`) REFERENCES `Variety` (`variety_id`),
  CONSTRAINT `cropplan_chk_1` CHECK ((`planned_area_acres` > 0)),
  CONSTRAINT `cropplan_chk_2` CHECK ((`expected_yield_kg` >= 0)),
  CONSTRAINT `cropplan_chk_3` CHECK ((`end_date` > `start_date`))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `Input` (
  `input_id` int NOT NULL AUTO_INCREMENT,
  `input_name` varchar(100) NOT NULL,
  `input_type` varchar(50) NOT NULL,
  `unit` varchar(20) NOT NULL,
  `unit_cost` decimal(10,2) NOT NULL,
  PRIMARY KEY (`input_id`),
  CONSTRAINT `input_chk_1` CHECK ((`unit_cost` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `Application` (
  `application_id` int NOT NULL AUTO_INCREMENT,
  `plan_id` int NOT NULL,
  `input_id` int NOT NULL,
  `application_date` date NOT NULL,
  `quantity_used` decimal(10,2) NOT NULL,
  `cost` decimal(10,2) NOT NULL,
  PRIMARY KEY (`application_id`),
  KEY `plan_id` (`plan_id`),
  KEY `input_id` (`input_id`),
  CONSTRAINT `application_ibfk_1`
    FOREIGN KEY (`plan_id`) REFERENCES `CropPlan` (`plan_id`) ON DELETE CASCADE,
  CONSTRAINT `application_ibfk_2`
    FOREIGN KEY (`input_id`) REFERENCES `Input` (`input_id`),
  CONSTRAINT `application_chk_1` CHECK ((`quantity_used` > 0)),
  CONSTRAINT `application_chk_2` CHECK ((`cost` >= 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `IrrigationEvent` (
  `irrigation_id` int NOT NULL AUTO_INCREMENT,

```

```

`plan_id` int NOT NULL,
`irrigation_date` date NOT NULL,
`duration_hours` decimal(4,1) NOT NULL,
`water_volume_litres` decimal(10,2) DEFAULT NULL,
`method` varchar(30) DEFAULT NULL,
PRIMARY KEY (`irrigation_id`),
KEY `plan_id` (`plan_id`),
CONSTRAINT `irrigationevent_ibfk_1`
  FOREIGN KEY (`plan_id`) REFERENCES `CropPlan` (`plan_id`) ON DELETE CASCADE,
CONSTRAINT `irrigationevent_chk_1` CHECK ((`duration_hours` > 0)),
CONSTRAINT `irrigationevent_chk_2` CHECK ((`water_volume_litres` >= 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

Listing 7.2: DDL for SoilTest, CropPlan, Input, Application and IrrigationEvent

```

CREATE TABLE `LabourActivity` (
  `labour_id` int NOT NULL AUTO_INCREMENT,
  `plan_id` int NOT NULL,
  `activity_date` date NOT NULL,
  `task_description` varchar(100) NOT NULL,
  `worker_count` int NOT NULL,
  `hours_worked` decimal(5,1) NOT NULL,
  `wage_per_hour` decimal(8,2) NOT NULL,
  PRIMARY KEY (`labour_id`),
  KEY `plan_id` (`plan_id`),
  CONSTRAINT `labouractivity_ibfk_1`
    FOREIGN KEY (`plan_id`) REFERENCES `CropPlan` (`plan_id`) ON DELETE CASCADE,
  CONSTRAINT `labouractivity_chk_1` CHECK ((`worker_count` > 0)),
  CONSTRAINT `labouractivity_chk_2` CHECK ((`hours_worked` > 0)),
  CONSTRAINT `labouractivity_chk_3` CHECK ((`wage_per_hour` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `Harvest` (
  `harvest_id` int NOT NULL AUTO_INCREMENT,
  `plan_id` int NOT NULL,
  `harvest_date` date NOT NULL,
  `quantity_kg` decimal(10,2) NOT NULL,
  `quality_grade` varchar(10) DEFAULT NULL,
  `storage_location` varchar(100) DEFAULT NULL,
  PRIMARY KEY (`harvest_id`),
  KEY `plan_id` (`plan_id`),
  CONSTRAINT `harvest_ibfk_1`
    FOREIGN KEY (`plan_id`) REFERENCES `CropPlan` (`plan_id`) ON DELETE CASCADE,
  CONSTRAINT `harvest_chk_1` CHECK ((`quantity_kg` >= 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `Buyer` (
  `buyer_id` int NOT NULL AUTO_INCREMENT,
  `buyer_name` varchar(100) NOT NULL,
  `contact_number` varchar(15) DEFAULT NULL,
  `location` varchar(150) DEFAULT NULL,
  PRIMARY KEY (`buyer_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `Sale` (
  `sale_id` int NOT NULL AUTO_INCREMENT,
  `harvest_id` int NOT NULL,
  `buyer_id` int NOT NULL,
  `sale_date` date NOT NULL,
  `quantity_sold_kg` decimal(10,2) NOT NULL,
  `price_per_kg` decimal(8,2) NOT NULL,
  `total_amount` decimal(12,2)
    GENERATED ALWAYS AS ((`quantity_sold_kg` * `price_per_kg`)) STORED,
  PRIMARY KEY (`sale_id`),
  KEY `harvest_id` (`harvest_id`),
  KEY `buyer_id` (`buyer_id`),
  CONSTRAINT `sale_ibfk_1`
    FOREIGN KEY (`harvest_id`) REFERENCES `Harvest` (`harvest_id`) ON DELETE CASCADE,
  CONSTRAINT `sale_ibfk_2`
    FOREIGN KEY (`buyer_id`) REFERENCES `Buyer` (`buyer_id`),
  CONSTRAINT `sale_chk_1` CHECK ((`quantity_sold_kg` > 0)),
  CONSTRAINT `sale_chk_2` CHECK ((`price_per_kg` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

Listing 7.3: DDL for LabourActivity, Harvest, Buyer and Sale

7.2 Constraints Summary

Type	Count
Foreign keys	13
CHECK constraints	19
UNIQUE keys	2 (Crop.crop_name, Variety(crop_id, variety_name))
Generated column	1 (Sale.total_amount)
Triggers	3
Views	1 (cropprofitability)

Table 7.1: Constraints Summary

7.3 Triggers

All three triggers fire BEFORE INSERT and raise SQLSTATE 45000. CHECK constraints see only one row, so cross-table rules need triggers.

Trigger	Table	Rule	Message
trg_cropplan_dates	CropPlan	Plan dates inside its Season dates	CropPlan dates must fall within the Season dates
trg_application_dates	Application	Application date inside the plan's start and end	Application date must fall within the Crop Plan duration
trg_sale_within_stock	Sale	Already-sold plus new quantity cannot exceed harvest quantity	Sale quantity exceeds available harvested stock

Table 7.2: Triggers

7.4 Data Manipulation Language (DML)

Sample data is inserted with INSERT statements in parent-to-child order. The sample data totals 46 rows (Table 7.3).

Crop Planning & Farm Input Management System

```
INSERT INTO Farm
VALUES
(1,'Green Valley Farm','Ravi Kumar','Nizamabad, Telangana',45.00),
(2,'Sunrise Agro','Lakshmi Devi','Warangal, Telangana',30.50);
```

```
INSERT INTO Field
VALUES
(1,1,'North Block',12.00,'Black Cotton'),
(2,1,'South Block',15.50,'Red Loamy'),
(3,2,'Riverside Plot',10.00,'Alluvial');
```

```
INSERT INTO Season
VALUES
(1,1,'Kharif 2026','2026-06-01','2026-10-31'),
(2,2,'Kharif 2026','2026-06-01','2026-10-31'),
(3,3,'Rabi 2025-26','2025-11-01','2026-03-31');
```

```
INSERT INTO Crop
VALUES
(1,'Maize','Cereal'),
(2,'Cotton','Fibre'),
(3,'Groundnut','Oilseed');
```

```
INSERT INTO Variety
VALUES
(1,1,'DHM-117',110),
(2,2,'Bt Cotton RCH-2',160),
(3,3,'TAG-24',100);
```

```
INSERT INTO SoilTest
VALUES
(1,1,'2026-05-15',6.8,'Medium','Low','High',
'Apply phosphorus-rich fertilizer before sowing'),
(2,2,'2026-05-16',7.2,'High','Medium','Medium',
'Standard NPK schedule sufficient'),
(3,3,'2025-10-20',6.5,'Low','Low','Medium',
'Apply nitrogen booster at sowing');
```

```
INSERT INTO CropPlan
VALUES
(1,1,1,12.00,'2026-06-10','2026-09-30',6000.00),
(2,2,2,15.50,'2026-06-15','2026-10-25',4500.00),
(3,3,3,10.00,'2025-11-10','2026-03-01',3000.00);
```

```
INSERT INTO Input
VALUES
(1,'Urea','Fertilizer','kg',25.00),
(2,'DAP','Fertilizer','kg',32.00),
(3,'Imidacloprid','Pesticide','litre',450.00),
(4,'Maize Seed','Seed','kg',180.00);
```

```
INSERT INTO Application
VALUES
(1,1,4,'2026-06-10',24.00,4320.00),
(2,1,1,'2026-06-25',100.00,2500.00),
(3,1,3,'2026-07-15',5.00,2250.00),
(4,2,2,'2026-06-20',150.00,4800.00),
(5,3,1,'2025-11-20',80.00,2000.00);
```

```
INSERT INTO IrrigationEvent
VALUES
```

```
(1,1,'2026-06-12',3.0,15000.00,'Sprinkler'),
(2,1,'2026-07-10',2.5,12000.00,'Sprinkler'),
(3,2,'2026-06-18',4.0,20000.00,'Drip'),
(4,3,'2025-11-25',3.5,16000.00,'Flood');

INSERT INTO LabourActivity
VALUES
(1,1,'2026-06-10','Sowing',5,6.0,60.00),
(2,1,'2026-08-20','Weeding',4,5.0,55.00),
(3,2,'2026-06-15','Sowing',6,7.0,65.00),
(4,3,'2025-11-10','Sowing',4,6.0,55.00);

INSERT INTO Harvest
VALUES
(1,1,'2026-09-28',6200.00,'A','Warehouse 1'),
(2,2,'2026-10-22',4300.00,'B','Warehouse 2'),
(3,3,'2026-02-28',2850.00,'A','Warehouse 1');

INSERT INTO Buyer
VALUES
(1,'Telangana Grain Traders','9876543210','Hyderabad'),
(2,'Cotton Corp of India','9123456780','Warangal'),
(3,'Local Oil Mill','9988776655','Nizamabad');

INSERT INTO Sale
(sale_id, harvest_id, buyer_id, sale_date, quantity_sold_kg, price_per_kg)
VALUES
(1,1,1,'2026-10-02',6000.00,18.50),
(2,2,2,'2026-10-25',4300.00,65.00),
(3,3,3,'2026-03-05',2800.00,55.00);
```

Listing 7.4: INSERT statements for sample data

Table	Rows	Table	Rows
Farm	2	Input	4
Field	3	Application	5
Season	3	IrrigationEvent	4
Crop	3	LabourActivity	4
Variety	3	Harvest	3
SoilTest	3	Buyer	3
CropPlan	3	Sale	3

Table 7.3: Sample Data Row Counts

7.5 Sample Outputs

SHOW FULL TABLES lists the 15 objects of the database: 14 tables and 1 view.

```
mysql>
[mysql> SHOW FULL TABLES;
+-----+-----+
| Tables_in_farm_management | Table_type |
+-----+-----+
| Application                | BASE TABLE |
| Buyer                      | BASE TABLE |
| Crop                      | BASE TABLE |
| CropPlan                   | BASE TABLE |
| cropprofitability          | VIEW        |
| Farm                      | BASE TABLE |
| Field                      | BASE TABLE |
| Harvest                    | BASE TABLE |
| Input                      | BASE TABLE |
| IrrigationEvent            | BASE TABLE |
| LabourActivity              | BASE TABLE |
| Sale                       | BASE TABLE |
| Season                     | BASE TABLE |
| SoilTest                   | BASE TABLE |
| Variety                    | BASE TABLE |
+-----+-----+
15 rows in set (0.001 sec)

mysql> █
```

Output 7.1: SHOW FULL TABLES

Crop Planning & Farm Input Management System

```
[mysql> SELECT * FROM Farm;
+-----+-----+-----+-----+-----+
| farm_id | farm_name      | owner_name | location                | total_area_acres |
+-----+-----+-----+-----+-----+
| 1       | Green Valley Farm | Ravi Kumar | Nizamabad, Telangana    | 45.00            |
| 2       | Sunrise Agro     | Lakshmi Devi | Warangal, Telangana     | 30.50            |
+-----+-----+-----+-----+-----+
2 rows in set (0.001 sec)

[mysql> SELECT * FROM Field;
+-----+-----+-----+-----+-----+
| field_id | farm_id | field_name | area_acres | soil_type |
+-----+-----+-----+-----+-----+
| 1        | 1       | North Block | 12.00      | Black Cotton |
| 2        | 1       | South Block | 15.50      | Red Loamy   |
| 3        | 2       | Riverside Plot | 10.00      | Alluvial    |
+-----+-----+-----+-----+-----+
3 rows in set (0.001 sec)

[mysql> SELECT * FROM Season;
+-----+-----+-----+-----+-----+
| season_id | field_id | season_name | start_date | end_date |
+-----+-----+-----+-----+-----+
| 1         | 1       | Kharif 2026 | 2026-06-01 | 2026-10-31 |
| 2         | 2       | Kharif 2026 | 2026-06-01 | 2026-10-31 |
| 3         | 3       | Rabi 2025-26 | 2025-11-01 | 2026-03-31 |
+-----+-----+-----+-----+-----+
3 rows in set (0.001 sec)

mysql>
```

Output 7.2: SELECT of Farm, Field and Season

```
[mysql> SELECT * FROM Crop;
+-----+-----+-----+
| crop_id | crop_name | category |
+-----+-----+-----+
| 1       | Maize     | Cereal   |
| 2       | Cotton    | Fibre    |
| 3       | Groundnut | Oilseed  |
+-----+-----+-----+
3 rows in set (0.001 sec)

[mysql> SELECT * FROM Variety;
+-----+-----+-----+-----+
| variety_id | crop_id | variety_name | maturity_days |
+-----+-----+-----+-----+
| 1          | 1       | DHM-117      | 110           |
| 2          | 2       | Bt Cotton RCH-2 | 160           |
| 3          | 3       | TAG-24       | 100           |
+-----+-----+-----+-----+
3 rows in set (0.000 sec)

[mysql> SELECT * FROM SoilTest;
+-----+-----+-----+-----+-----+-----+-----+-----+
| soil_test_id | field_id | test_date | ph_level | nitrogen_level | phosphorus_level | potassium_level | recommendation |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1            | 1       | 2026-05-15 | 6.8      | Medium         | Low              | High           | Apply phosphorus-rich fertilizer before sowing |
| 2            | 2       | 2026-05-16 | 7.2      | High           | Medium           | Medium         | Standard NPK schedule sufficient |
| 3            | 3       | 2025-10-20 | 6.5      | Low            | Low              | Medium         | Apply nitrogen booster at sowing |
+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

[mysql> SELECT * FROM Input;
+-----+-----+-----+-----+-----+
| input_id | input_name | input_type | unit | unit_cost |
+-----+-----+-----+-----+-----+
| 1        | Urea       | Fertilizer | kg   | 25.00     |
| 2        | DAP        | Fertilizer | kg   | 32.00     |
| 3        | Imidacloprid | Pesticide | litre | 450.00    |
| 4        | Maize Seed | Seed       | kg   | 180.00    |
+-----+-----+-----+-----+-----+
4 rows in set (0.000 sec)

mysql>
```

Output 7.3: SELECT of Crop, Variety, SoilTest and Input

```

mysql> SELECT * FROM CropPlan;
+-----+-----+-----+-----+-----+-----+-----+
| plan_id | season_id | variety_id | planned_area_acres | start_date | end_date | expected_yield_kg |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 1 | 12.00 | 2026-06-10 | 2026-09-30 | 6000.00 |
| 2 | 2 | 2 | 15.50 | 2026-06-15 | 2026-10-25 | 4500.00 |
| 3 | 3 | 3 | 10.00 | 2025-11-10 | 2026-03-01 | 3000.00 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.001 sec)

mysql> SELECT * FROM Application;
+-----+-----+-----+-----+-----+-----+
| application_id | plan_id | input_id | application_date | quantity_used | cost |
+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 4 | 2026-06-10 | 24.00 | 4320.00 |
| 2 | 1 | 1 | 2026-06-25 | 100.00 | 2500.00 |
| 3 | 1 | 3 | 2026-07-15 | 5.00 | 2250.00 |
| 4 | 2 | 2 | 2026-06-20 | 150.00 | 4800.00 |
| 5 | 3 | 1 | 2025-11-20 | 80.00 | 2000.00 |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.001 sec)

mysql> SELECT * FROM IrrigationEvent;
+-----+-----+-----+-----+-----+-----+
| irrigation_id | plan_id | irrigation_date | duration_hours | water_volume_litres | method |
+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 2026-06-12 | 3.0 | 15000.00 | Sprinkler |
| 2 | 1 | 2026-07-10 | 2.5 | 12000.00 | Sprinkler |
| 3 | 2 | 2026-06-18 | 4.0 | 20000.00 | Drip |
| 4 | 3 | 2025-11-25 | 3.5 | 16000.00 | Flood |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.001 sec)

mysql> SELECT * FROM LabourActivity;
+-----+-----+-----+-----+-----+-----+-----+
| labour_id | plan_id | activity_date | task_description | worker_count | hours_worked | wage_per_hour |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 2026-06-10 | Sowing | 5 | 6.0 | 60.00 |
| 2 | 1 | 2026-08-20 | Weeding | 4 | 5.0 | 55.00 |
| 3 | 2 | 2026-06-15 | Sowing | 6 | 7.0 | 65.00 |
| 4 | 3 | 2025-11-10 | Sowing | 4 | 6.0 | 55.00 |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.001 sec)

mysql>

```

Output 7.4: SELECT of CropPlan, Application, IrrigationEvent and LabourActivity

```
mysql> SELECT * FROM Harvest;
+-----+-----+-----+-----+-----+-----+
| harvest_id | plan_id | harvest_date | quantity_kg | quality_grade | storage_location |
+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 2026-09-28 | 6200.00 | A | Warehouse 1 |
| 2 | 2 | 2026-10-22 | 4300.00 | B | Warehouse 2 |
| 3 | 3 | 2026-02-28 | 2850.00 | A | Warehouse 1 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.001 sec)

mysql> SELECT * FROM Buyer;
+-----+-----+-----+-----+
| buyer_id | buyer_name | contact_number | location |
+-----+-----+-----+-----+
| 1 | Telangana Grain Traders | 9876543210 | Hyderabad |
| 2 | Cotton Corp of India | 9123456780 | Warangal |
| 3 | Local Oil Mill | 9988776655 | Nizamabad |
+-----+-----+-----+-----+
3 rows in set (0.000 sec)

mysql> SELECT * FROM Sale;
+-----+-----+-----+-----+-----+-----+-----+
| sale_id | harvest_id | buyer_id | sale_date | quantity_sold_kg | price_per_kg | total_amount |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 1 | 2026-10-02 | 6000.00 | 18.50 | 111000.00 |
| 2 | 2 | 2 | 2026-10-25 | 4300.00 | 65.00 | 279500.00 |
| 3 | 3 | 3 | 2026-03-05 | 2800.00 | 55.00 | 154000.00 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.001 sec)

mysql>
```

Output 7.5: SELECT of Harvest, Buyer and Sale

8. QUERIES WITH OUTPUTS

This chapter lists the SQL queries and their outputs on the sample data.

Query 1: Full field plan (CropPlan joined to Season, Field, Farm, Variety and Crop)

```
SELECT
    cp.plan_id,
    f.farm_name,
    f.location,
    fi.field_name,
    s.season_name,
    c.crop_name,
    v.variety_name,
    cp.start_date,
    cp.end_date,
    cp.expected_yield_kg
FROM CropPlan cp
JOIN Season s
    ON cp.season_id = s.season_id
JOIN Field fi
    ON s.field_id = fi.field_id
JOIN Farm f
    ON fi.farm_id = f.farm_id
JOIN Variety v
    ON cp.variety_id = v.variety_id
JOIN Crop c
    ON v.crop_id = c.crop_id
ORDER BY cp.plan_id;
```

The query returns 3 rows: Maize, Cotton and Groundnut, with farm, field, season, dates and expected yield.

```
mysql> SELECT
->     cp.plan_id,
->     f.farm_name,
->     f.location,
->     fi.field_name,
->     s.season_name,
->     c.crop_name,
->     v.variety_name,
->     cp.start_date,
->     cp.end_date,
->     cp.expected_yield_kg
-> FROM CropPlan cp
-> JOIN Season s
->     ON cp.season_id = s.season_id
-> JOIN Field fi
->     ON s.field_id = fi.field_id
-> JOIN Farm f
->     ON fi.farm_id = f.farm_id
-> JOIN Variety v
->     ON cp.variety_id = v.variety_id
-> JOIN Crop c
->     ON v.crop_id = c.crop_id
-> ORDER BY cp.plan_id;
```

plan_id	farm_name	location	field_name	season_name	crop_name	variety_name	start_date	end_date	expected_yield_kg
1	Green Valley Farm	Nizamabad, Telangana	North Block	Kharif 2026	Maize	DHM-117	2026-06-10	2026-09-30	6000.00
2	Green Valley Farm	Nizamabad, Telangana	South Block	Kharif 2026	Cotton	Bt Cotton RCH-2	2026-06-15	2026-10-25	4500.00
3	Sunrise Agro	Warangal, Telangana	Riverside Plot	Rabi 2025-26	Groundnut	TAG-24	2025-11-10	2026-03-01	3000.00

3 rows in set (0.003 sec)

```
mysql>
```

Output 8.1: Full field plan

Query 2: Input cost per CropPlan (join, SUM, GROUP BY on Application.cost)

```
SELECT
    c.crop_name AS Crop,
    SUM(a.cost) AS `Total input cost`
FROM CropPlan cp
JOIN Variety v
    ON cp.variety_id = v.variety_id
JOIN Crop c
    ON v.crop_id = c.crop_id
JOIN Application a
    ON cp.plan_id = a.plan_id
GROUP BY c.crop_name
ORDER BY cp.plan_id;
```

Crop	Total input cost
Maize	9070.00
Cotton	4800.00
Groundnut	2000.00

Output 8.2: Input cost per crop plan

Query 3: Yield exceptions (correlated subquery)

```
SELECT
    cp.plan_id AS Plan,
    c.crop_name AS Crop,
    cp.expected_yield_kg AS `Expected yield (kg)`,
    (
        SELECT SUM(h.quantity_kg)
        FROM Harvest h
        WHERE h.plan_id = cp.plan_id
    ) AS `Actual yield (kg)`
FROM CropPlan cp
JOIN Variety v
    ON cp.variety_id = v.variety_id
JOIN Crop c
    ON v.crop_id = c.crop_id
WHERE (
    SELECT SUM(h.quantity_kg)
    FROM Harvest h
    WHERE h.plan_id = cp.plan_id
) > cp.expected_yield_kg;
```

The query returns plan 1 (Maize), where the actual yield exceeded the expected yield.

Plan	Crop	Expected yield (kg)	Actual yield (kg)
1	Maize	6000.00	6200.00

Output 8.3: Yield exceptions

Query 4: cropprofitability view (LEFT JOINS on derived tables, COALESCE)

```
SHOW CREATE VIEW cropprofitability;
```

And then:

```
SELECT * FROM cropprofitability;
```

Plan	Crop	Input cost	Labour cost	Sale revenue	Net profit
1	Maize	9070.00	2900.00	111000.00	99030.00
2	Cotton	4800.00	2730.00	279500.00	271970.00
3	Groundnut	2000.00	1320.00	154000.00	150680.00

Output 8.4: cropprofitability view

Net profit is sale revenue minus input cost and labour cost for each crop plan.

Query 5: Remaining stock per harvest

```
SELECT
  h.harvest_id,
  h.quantity_kg AS `Harvested (kg)`,
  COALESCE(SUM(s.quantity_sold_kg), 0) AS `Sold (kg)`,
  h.quantity_kg - COALESCE(SUM(s.quantity_sold_kg), 0) AS `Remaining stock (kg)`
FROM Harvest h
LEFT JOIN Sale s
  ON h.harvest_id = s.harvest_id
GROUP BY h.harvest_id, h.quantity_kg
ORDER BY h.harvest_id;
```

The remaining stock for the three harvests is 200, 0 and 50 kg.

```
mysql> SELECT
->   h.harvest_id,
->   h.quantity_kg AS `Harvested (kg)`,
->   COALESCE(SUM(s.quantity_sold_kg), 0) AS `Sold (kg)`,
->   h.quantity_kg - COALESCE(SUM(s.quantity_sold_kg), 0) AS `Remaining stock (kg)`
-> FROM Harvest h
-> LEFT JOIN Sale s
->   ON h.harvest_id = s.harvest_id
-> GROUP BY h.harvest_id, h.quantity_kg
-> ORDER BY h.harvest_id;
+-----+-----+-----+-----+
| harvest_id | Harvested (kg) | Sold (kg) | Remaining stock (kg) |
+-----+-----+-----+-----+
| 1 | 6200.00 | 6000.00 | 200.00 |
| 2 | 4300.00 | 4300.00 | 0.00 |
| 3 | 2850.00 | 2800.00 | 50.00 |
+-----+-----+-----+-----+
3 rows in set (0.005 sec)

mysql>
```

Output 8.5: Remaining stock per harvest

Query 6: Revenue per buyer

```
SELECT
    b.buyer_id,
    b.buyer_name AS Buyer,
    SUM(s.quantity_sold_kg * s.price_per_kg) AS `Revenue`
FROM Buyer b
JOIN Sale s
    ON b.buyer_id = s.buyer_id
GROUP BY b.buyer_id, b.buyer_name
ORDER BY b.buyer_id;
```

The revenue per buyer is 111,000, 279,500 and 154,000.

```
mysql> SELECT
->     b.buyer_id,
->     b.buyer_name AS Buyer,
->     SUM(s.quantity_sold_kg * s.price_per_kg) AS `Revenue`
-> FROM Buyer b
-> JOIN Sale s
->     ON b.buyer_id = s.buyer_id
-> GROUP BY b.buyer_id, b.buyer_name
[ -> ORDER BY b.buyer_id;
```

buyer_id	Buyer	Revenue
1	Telangana Grain Traders	111000.0000
2	Cotton Corp of India	279500.0000
3	Local Oil Mill	154000.0000

3 rows in set (0.003 sec)

```
mysql> █
```

Output 8.6: Revenue per buyer

9. UI DESIGN AND SCREENSHOTS

9.1 Overview

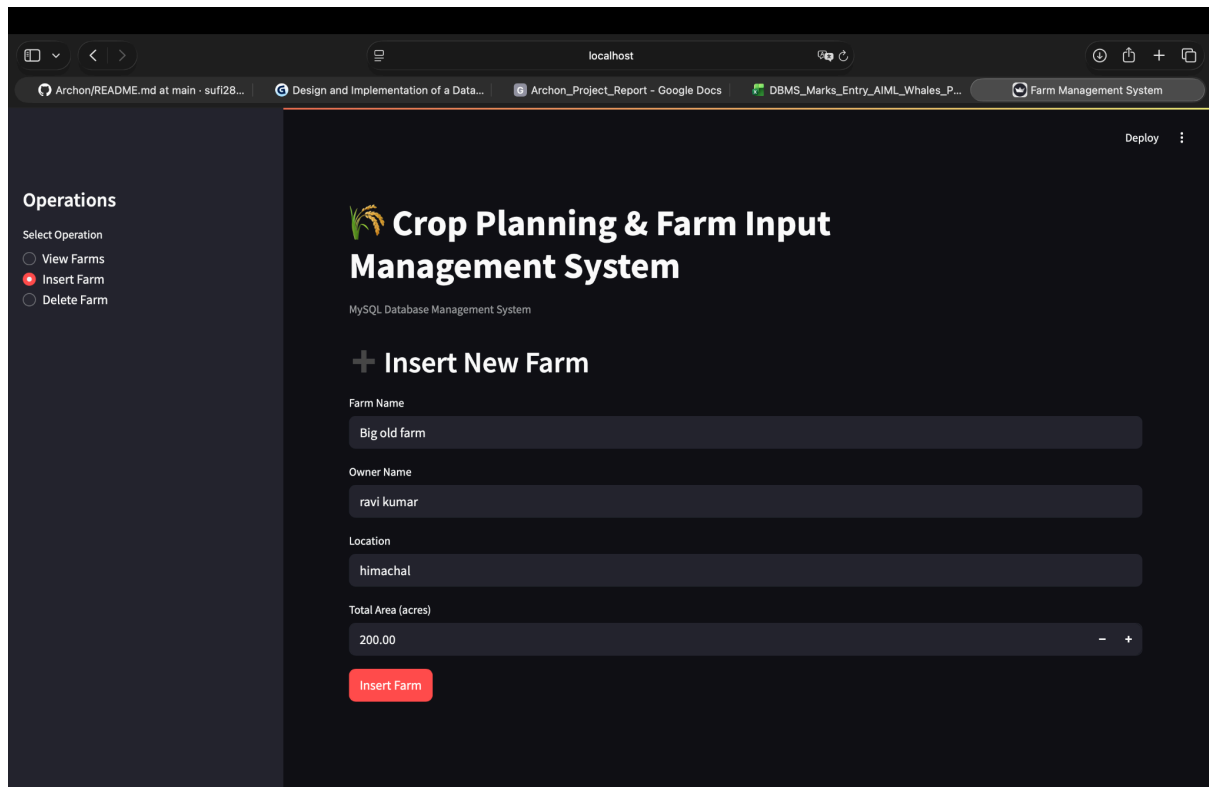
A local Streamlit web app (Python, mysql-connector-python, python-dotenv) connected to the farm_management database. It demonstrates three operations on the Farm entity.

9.2 View Farm Records



Figure 9.1: View Farm Records

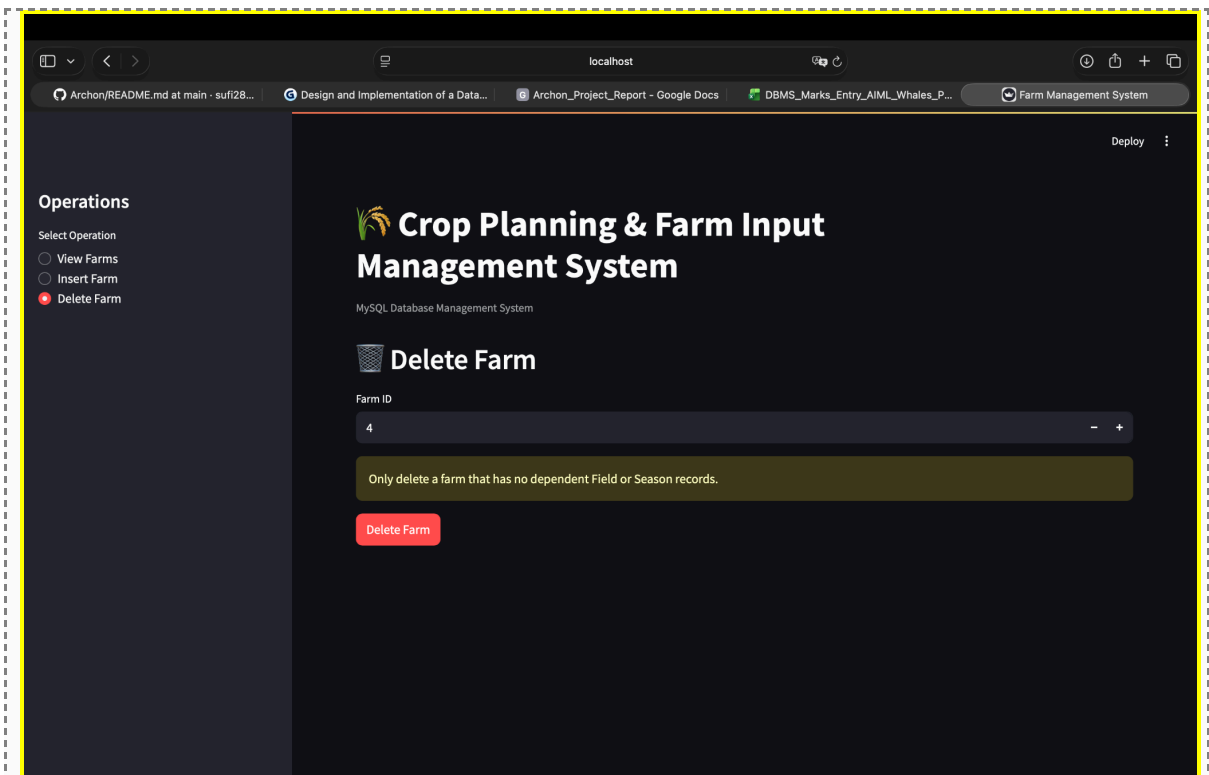
9.3 Insert a Farm Record



The screenshot shows a web application interface for 'Crop Planning & Farm Input Management System'. On the left, there is a sidebar with 'Operations' and a 'Select Operation' menu where 'Insert Farm' is selected. The main content area displays the title 'Crop Planning & Farm Input Management System' and 'MySQL Database Management System'. Below this, there is a section titled '+ Insert New Farm'. The form contains the following fields: 'Farm Name' (text input with 'Big old farm'), 'Owner Name' (text input with 'ravi kumar'), 'Location' (text input with 'himachal'), and 'Total Area (acres)' (text input with '200.00' and increment/decrement buttons). A red 'Insert Farm' button is at the bottom of the form.

Figure 9.2: Insert a Farm Record

9.4 Delete a Farm Record



The screenshot shows the 'Delete Farm' form in the same web application. The sidebar now has 'Delete Farm' selected. The main content area displays the title 'Crop Planning & Farm Input Management System' and 'MySQL Database Management System'. Below this, there is a section titled 'Delete Farm'. The form contains a 'Farm ID' field (text input with '4' and increment/decrement buttons). A warning message states: 'Only delete a farm that has no dependent Field or Season records.' A red 'Delete Farm' button is at the bottom of the form.

Figure 9.3: Delete a Farm Record

Crop Planning & Farm Input Management System

Note: because of ON DELETE CASCADE, deleting a Farm that has fields also removes its dependent data. For the demo, a newly inserted farm is deleted.

10. IMPLEMENTATION DETAILS

10.1 Technology Stack

Layer	Technology
Database	MySQL 9.7.1, database farm_management, 14 tables, 1 view
Query language	SQL (DDL, DML, triggers, views)
Front end / back end	Python, Streamlit, mysql-connector-python, python-dotenv
Version control	Git, GitHub

Table 10.1: Technology Stack

10.2 Database Connectivity

The app reads the host, user, password and database name from environment variables loaded from a .env file (not committed) and connects with mysql-connector-python.

```
import os
import mysql.connector
from dotenv import load_dotenv

load_dotenv()

def get_connection():
    return mysql.connector.connect(
        host=os.getenv("DB_HOST", "localhost"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
        database=os.getenv("DB_NAME", "farm_management")
    )
```

Listing 10.1: Database connection code

The application loads database credentials from environment variables using `python-dotenv`. The `get_connection()` function establishes a connection to the MySQL `farm_management` database using `mysql-connector-python`. Database credentials are stored in a local `.env` file and are not committed to the GitHub repository.

10.3 Key Code Snippets

The definition of the Application table, which resolves the many-to-many relationship between CropPlan and Input:

```
CREATE TABLE `Application` (
  `application_id` int NOT NULL AUTO_INCREMENT,
  `plan_id` int NOT NULL,
  `input_id` int NOT NULL,
  `application_date` date NOT NULL,
  `quantity_used` decimal(10,2) NOT NULL,
  `cost` decimal(10,2) NOT NULL,
  PRIMARY KEY (`application_id`),
  KEY `plan_id` (`plan_id`),
  KEY `input_id` (`input_id`),
  CONSTRAINT `application_ibfk_1`
    FOREIGN KEY (`plan_id`) REFERENCES `CropPlan` (`plan_id`)
    ON DELETE CASCADE,
  CONSTRAINT `application_ibfk_2`
    FOREIGN KEY (`input_id`) REFERENCES `Input` (`input_id`),
  CONSTRAINT `application_chk_1`
    CHECK ((`quantity_used` > 0)),
  CONSTRAINT `application_chk_2`
    CHECK ((`cost` >= 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_0900_ai_ci;
```

Listing 10.2: Application table

The trigger that stops a sale from exceeding the harvested stock:

```
DELIMITER $$

CREATE TRIGGER `trg_sale_within_stock`
BEFORE INSERT ON `Sale`
FOR EACH ROW
BEGIN
  DECLARE harvested DECIMAL(10,2);
  DECLARE already_sold DECIMAL(10,2);

  SELECT quantity_kg
  INTO harvested
  FROM Harvest
  WHERE harvest_id = NEW.harvest_id;

  SELECT IFNULL(SUM(quantity_sold_kg),0)
  INTO already_sold
  FROM Sale
  WHERE harvest_id = NEW.harvest_id;

  IF (already_sold + NEW.quantity_sold_kg) > harvested THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE TEXT =
      'Sale quantity exceeds available harvested stock';
  END IF;
END$$

DELIMITER ;
```

Listing 10.3: Sale stock trigger

The cropprofitability view:

```
CREATE VIEW `cropprofitability` AS
SELECT
  cp.plan_id,
  c.crop_name,
  v.variety_name,
  COALESCE(input_cost.total, 0) AS total_input_cost,
  COALESCE(labour_cost.total, 0) AS total_labour_cost,
  COALESCE(sale_rev.total, 0) AS total_sale_revenue,
  (
    COALESCE(sale_rev.total, 0)
    - COALESCE(input_cost.total, 0)
    - COALESCE(labour_cost.total, 0)
  ) AS net_profit
FROM CropPlan cp
JOIN Variety v
  ON cp.variety_id = v.variety_id
JOIN Crop c
  ON v.crop_id = c.crop_id
LEFT JOIN (
  SELECT
    plan_id,
    SUM(cost) AS total
  FROM Application
  GROUP BY plan_id
) input_cost
  ON input_cost.plan_id = cp.plan_id
LEFT JOIN (
  SELECT
    plan_id,
    SUM(worker_count * hours_worked * wage_per_hour) AS total
  FROM LabourActivity
  GROUP BY plan_id
) labour_cost
  ON labour_cost.plan_id = cp.plan_id
LEFT JOIN (
  SELECT
    h.plan_id,
    SUM(s.quantity_sold_kg * s.price_per_kg) AS total
  FROM Harvest h
  JOIN Sale s
    ON s.harvest_id = h.harvest_id
  GROUP BY h.plan_id
) sale_rev
  ON sale_rev.plan_id = cp.plan_id;
```

Listing 10.4: Crop profitability view

The correlated subquery for yield exceptions:

```
SELECT
  cp.plan_id AS Plan,
  c.crop_name AS Crop,
  cp.expected_yield_kg AS `Expected yield (kg)`,
  (
    SELECT SUM(h.quantity_kg)
    FROM Harvest h
    WHERE h.plan_id = cp.plan_id
  ) AS `Actual yield (kg)`
FROM CropPlan cp
JOIN Variety v
  ON cp.variety_id = v.variety_id
JOIN Crop c
  ON v.crop_id = c.crop_id
WHERE (
  SELECT SUM(h.quantity_kg)
  FROM Harvest h
  WHERE h.plan_id = cp.plan_id
) > cp.expected_yield_kg;
```

Listing 10.5: Yield-exceptions query

10.4 Integrity Features

- **Referential integrity:** 13 foreign keys, with CASCADE for dependent data and RESTRICT for shared catalog and buyer data.
- **Validation:** 19 CHECK constraints, 2 UNIQUE keys, NOT NULL on mandatory fields.
- **Business rules:** 3 triggers for date windows and sale stock limits.
- **Consistency:** generated total_amount; no redundancy beyond the deliberate cost snapshot.

11. TESTING

Testing covers structure creation, data loading, query results, constraint and trigger enforcement, and the user interface. Each test case lists the expected result and the actual result returned by the MySQL server.

11.1 Test Cases and Results

ID	Test case	Expected result	Actual result	Status	
T1	SHOW FULL TABLES	15 rows (14 tables, 1 view)	15 rows returned (14 tables, 1 view)	pass	
T2	Row counts after load	46 rows total	46 rows present across 14 tables	pass	
T3–T6	Queries 1–4	Results as in Chapter 8	Results matched the outputs shown in Chapter 8	pass	
T7	Duplicate crop_name	ERROR 1062	ERROR 1062 – Duplicate entry for crop_name	pass	
T8	Duplicate (crop_id, variety_name)	ERROR 1062	ERROR 1062 – Duplicate entry for (crop_id, variety_name)	pass	
T9	Field with non-existent farm_id	ERROR 1452	ERROR 1452 – Cannot add or update a child row	pass	
T10	Farm with NULL farm_name	ERROR 1048	ERROR 1048 – Column farm_name cannot be null	pass	
T11	Farm with total_area_acres ≤ 0	ERROR 3819 (CHECK)	ERROR 3819 – CHECK constraint violated	pass	
T12	ph_level = 15	ERROR 3819	ERROR 3819 – CHECK constraint violated	pass	

ID	Test case	Expected result	Actual result	Status	
T13	Season with end $\leq$ start	ERROR 3819	ERROR 3819 – CHECK constraint violated	pass	
T14	CropPlan outside Season dates	ERROR 1644, date message	ERROR 1644 – CropPlan dates must fall within the Season dates	pass	
T15	Application outside plan window	ERROR 1644	ERROR 1644 – Application date must fall within the Crop Plan duration	pass	
T16	Sale of 300 kg on harvest 1 (200 kg left)	ERROR 1644	ERROR 1644 – Sale quantity exceeds available harvested stock	pass	
T17	Delete an Input used in Application	ERROR 1451	ERROR 1451 – Cannot delete/update a parent row	pass	
T18	Delete a Buyer with sales	ERROR 1451	ERROR 1451 – Cannot delete/update a parent row	pass	
T19	Delete a Variety used in CropPlan	ERROR 1451	ERROR 1451 – Cannot delete/update a parent row	pass	
T20	Delete a Farm (in a transaction, then ROLLBACK)	Cascades to child rows	Child rows cascaded successfully; transaction rolled back	pass	

ID	Test case	Expected result	Actual result	Status	
T21	Insert a Sale; read total_amount	Computed automatically	total_amount computed automatically from quantity_sold_kg × price_per_kg	pass	
T22	UI: view Farms	Rows displayed	Farm records displayed successfully in the UI	pass	
T23	UI: insert Farm	Row added	New Farm row added successfully	pass	
T24	UI: delete Farm	Row removed	Farm row removed successfully	pass	

Table 11.1: Test Cases and Results

11.2 Observations

The database rejects invalid data and enforces the cross-table rules. One known gap: triggers fire on INSERT only, so an UPDATE can bypass them. This is listed under future enhancements.

12. CONCLUSION AND FUTURE ENHANCEMENTS

12.1 Conclusion

This project designed and implemented a 14-table, 3NF relational database for crop planning and farm input management. It enforces integrity with keys, CHECK constraints, a generated column and triggers, reports profitability through a view, and was verified with queries and tests. A Streamlit prototype demonstrates the database through a working UI.

12.2 Future Enhancements

- BEFORE UPDATE triggers so updates cannot bypass the date and stock rules
- Date checks for irrigation and labour records
- Area-consistency rules (fields within farm, plan within field)
- A full CRUD UI for every entity
- Authentication and role-based access control
- Deployment beyond localhost
- More reporting views, indexes and regular backups

13. REFERENCES

- [1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.
- [2] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill, 2019.
- [3] MySQL Reference Manual, Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/>
- [4] E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970.
- [5] P. P. Chen, "The entity-relationship model: Toward a unified view of data," ACM Transactions on Database Systems, vol. 1, no. 1, pp. 9–36, 1976.
- [6] Streamlit documentation. [Online]. Available: <https://docs.streamlit.io/>
- [7] MySQL Connector/Python documentation. [Online]. Available: <https://dev.mysql.com/doc/connector-python/>

APPENDIX: GITHUB REPOSITORY LINK

The SQL scripts, documentation and project files are available at:

